Lab 9: Multi-Body Browser Simulation

Name: Aidan Siddiqi Date: 11/1/2026 Lab: PHYS 311 Module 9

AI use disclaimer: AI (Claude) was used in preparing this code and write-up for coding assistance, tidying up code, and improving code flow/organization. This is not to say that there was no human work done; all of it included input and assistance from myself and many iterations and human attempts are consolidated into what is seen here.

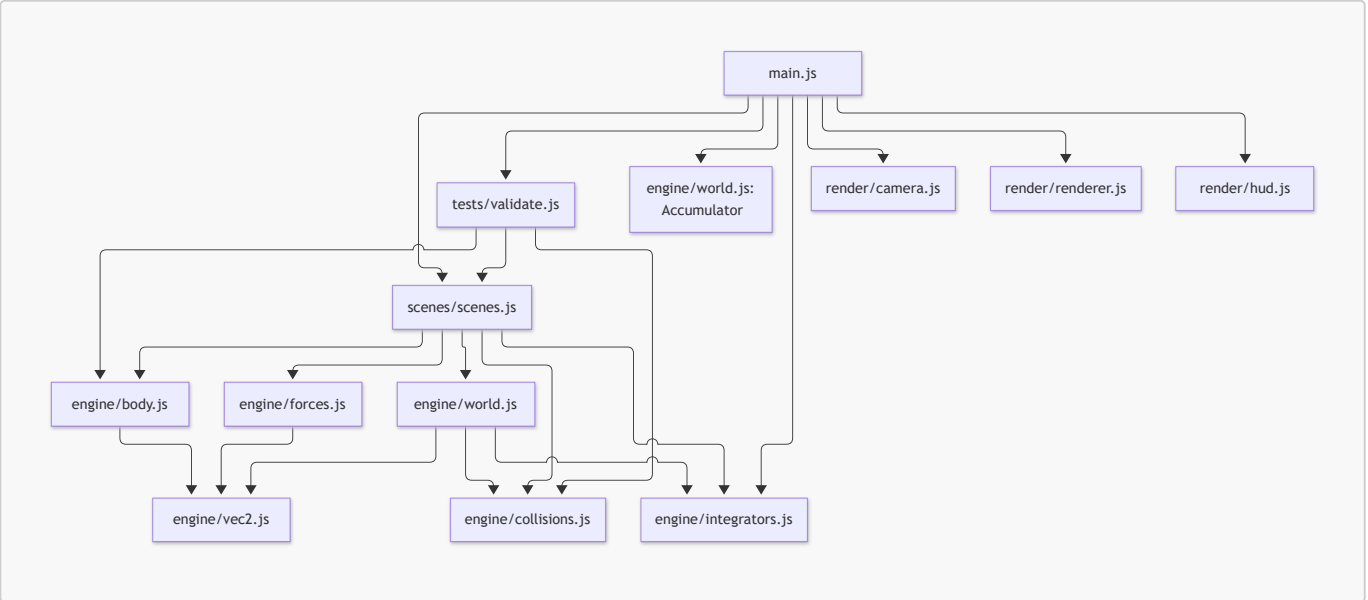
Live link: https://asiddiqi5.github.io/phys311/labs/lab09/

The structure follows the Week 9 reference engine.

Bonus attempted: spatial grid broad phase.

All numbers below come from `node tests/validate.js` and the in-page buttons, which give identical digits.

Module Dependency Diagram



No arrow goes from `engine/` into `render/`. Only `main.js` imports both. A search of `engine/` for `ctx`, `canvas`, `document`, `window` and `requestAnimationFrame` finds nothing, and the whole engine runs in Node.

Part A: The Modular Engine

`World.step(dt)` runs the five stages, including clear forces, apply forces, integrate, collide (walls, then pairs), advance time. Stages 1 and 2 are `computeForces()`, because Velocity Verlet calls it again.

There are four forces with `apply(world)`, including Gravity, Drag, Spring and NewtonGravity. The conservative ones also have `potentialEnergy(world)`. Internal forces are applied as `a.applyForce(F)` and `b.applyForce(F.neg())`, so momentum conservation is built into the forces.

The integrators (Euler, Euler-Cromer, Velocity Verlet) share `step(world, dt)`. Switching is one line, `world.integrator = new VelocityVerlet()`, and the dropdown does it live.

The HUD compares against `world.baseline`, which `main.js` captures once when the scene is built. Capturing it inside the frame loop would make the drift read zero forever.

Part B1: Collision Pipeline and the Jitter Bug

`BoxBounds` clamps a body back inside the walls, then reflects the outward velocity with factor e. `resolveContact` finds the normal and the closing speed vn. If the pair is separating, it pushes them apart. Otherwise, it applies the impulse and then pushes them away from each other. The push adds no momentum. The ball pit uses five unmoving pegs.

Jitter Demo: Ball pit, 100 balls, e = 0.6. I let it settle for 20 seconds, then averaged KE over the next 2 seconds.


Scene
Simulation
1b - Ball pit (gravity, static pegs)
Integrator
Euler-Cromer
Bodies
100
Restitution e = 0.60
Fixed timestep DT
1/240 s (default)
Broad phase
Brute force, O(N²)
Pause
Step
Reset
separating guard (vn > 0)
trails
v & a
grid
Gravity, light drag and five static pegs. Energy is supposed to drop here. Let the pile settle, then uncheck the separating guard and watch KE in the HUD.
Conservation HUD
t 20.10 s
bodies 105
integrator Euler-Cromer
collisions 677386
KE 0.4072 J
PE 1002.2532 J
E 1002.6604 J
ΔE/E₀ -87.3227 %
|p| 5.263290
|Δp|/|p₀| 9.76e+0
|Δ(p - Jext)|/|p₀| n/a (external field)
fps 60
physics 0.73 ms/frame
render 0.05 ms/frame
pair checks 5460 /step
steps/frame 4
Headless checks
Same code as node tests/validate.js . It needs no canvas.
Run 13 checks
Scaling benchmark



Separating Guard	Mean KE, t = 20 to 22 s	Fastest Ball
On	0.429 J	0.27 m/s
Off	1.079 J	0.37 m/s

The two still images look almost the same, so we need to look at the KE. A pile that should be resting has 2.5 times more KE without the guard. Without it, a pair that overlaps but is already moving apart gets another impulse. That flips their relative velocity and sends them back into each other, so the pile keeps buzzing. With the guard, a separating pair only gets pushed out.

Part B2: Conservation HUD and Validation

Momentum and Energy Tests: 20 balls in a box, no gravty, DT = 1/240 s, 60 s.

Condition	Pair Collisions	Wall Hits	|p| at t = 0	|p| at t = 60 s	|p − J_ext| at t = 0	|p − J_ext| at t = 60 s
e = 1	295	344	20.704000742217	23.149931248554	20.704000742217	20.704000742217
e = 0.8	92	126	20.704000742217	1.960526381110	20.704000742217	20.704000742217

Condition	E at t = 0	E at t = 60 s	ΔE/E0	Steps Where E Rose
e = 1	134.158845344895 J	134.158845344894 J	-1.7e-15	0
e = 0.8	134.158845344895 J	2.979713778530 J	-97.8 %	0

Why |p| changes?: The lesson says |p| should stay constant in the box. Plain |p| cannot, because the walls push on the balls. A wall is an outside force. So, the engine adds up the momentum the walls give, J_ext, and checks p − J_ext instead. It holds to 1.5e-15 (e = 1) and 1.7e-16 (e = 0.8) relative change.

To check plain |p| as well, I turned the walls off and started the balls bunched together:

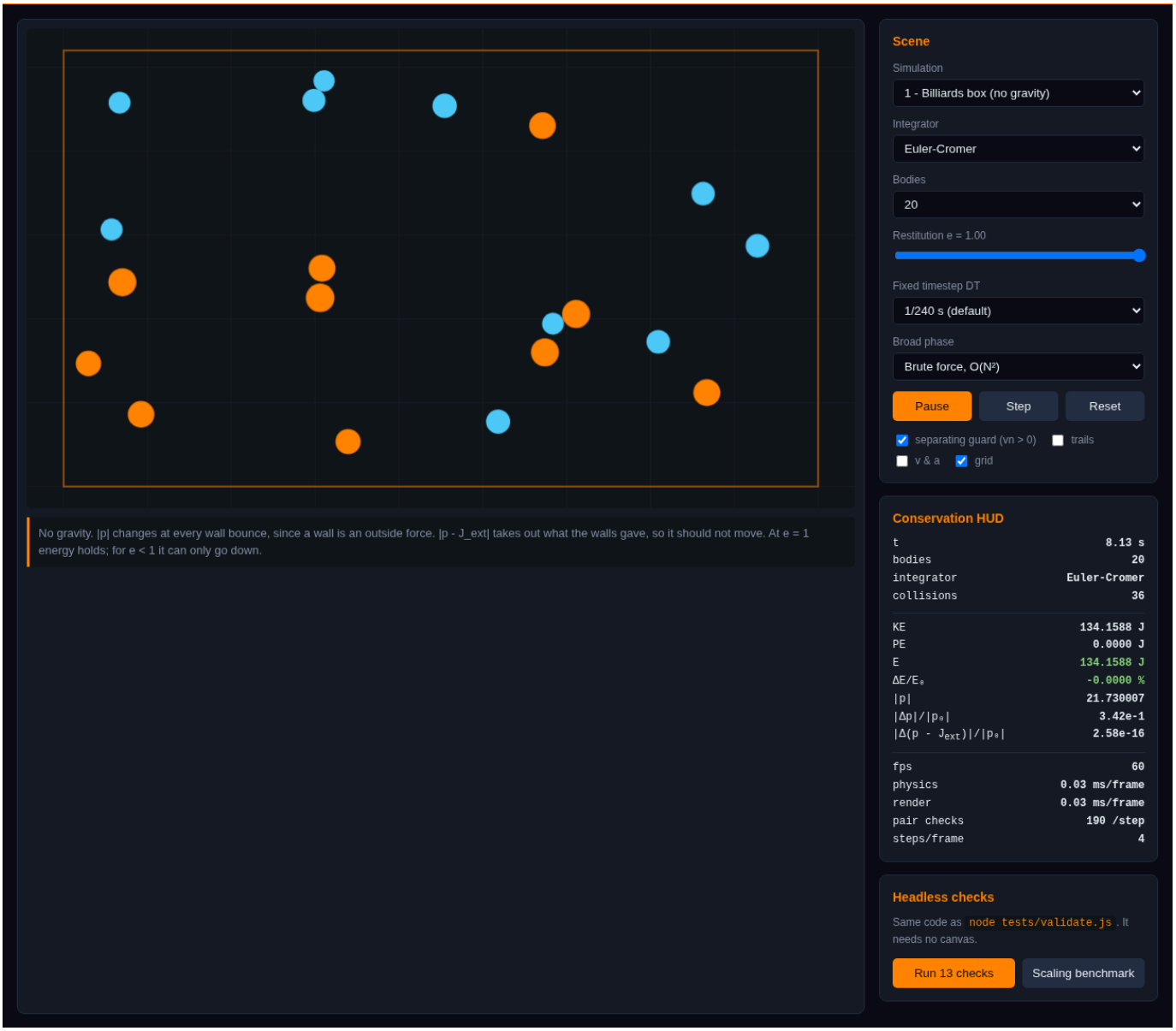
e	|Δp|/|p0| after 10 s	KE Kept	Collisions
1.0	2.3e-16	100.00 %	12
0.8	1.7e-16	87.90 %	14
0.5	4.3e-17	77.30 %	15
0.0	1.1e-17	75.56 %	13

Why do they behave differently?: Every pair impulse is equal and opposite, so it cancels in the total momentum for e. Energy is only kept at e = 1. For e < 1, each collision removes part of the KE, and nothing adds it back. So, E falls and never rises, while p − J_ext does not move.

Part C: Three Scenes from One Engine

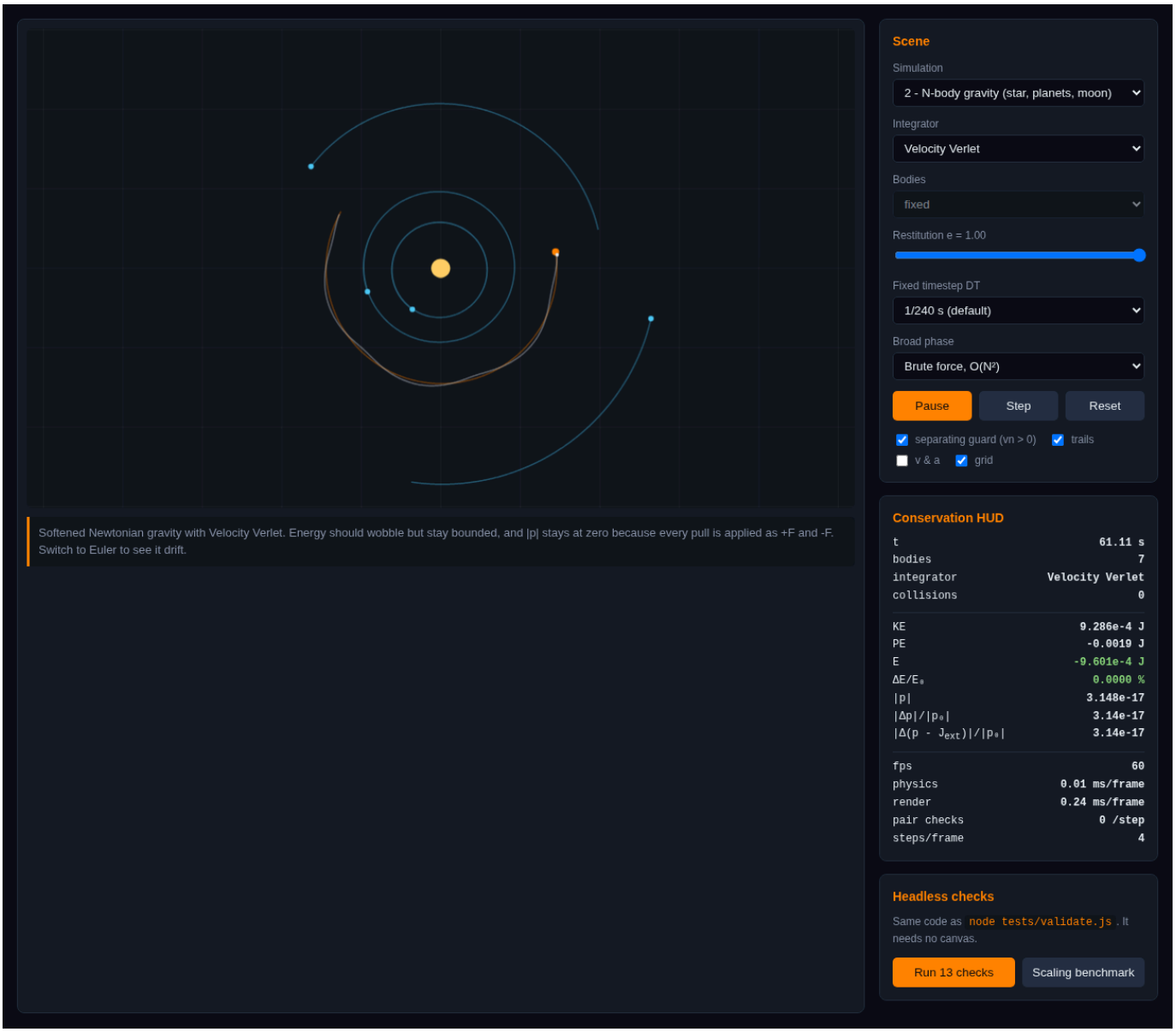
All scenes are in `scenes/scenes.js`. The dropdown switches them at runtime. The integrator switch, e slider and HUD work in every scene.

Scene 1: Billiards Box



20 to 500 balls with masses from 1 to 2 kg. Diagnostic: p − J_ext stays fixed, and E falls monotonically for e < 1 (tables above).

Scene 2: Gravitational N-body

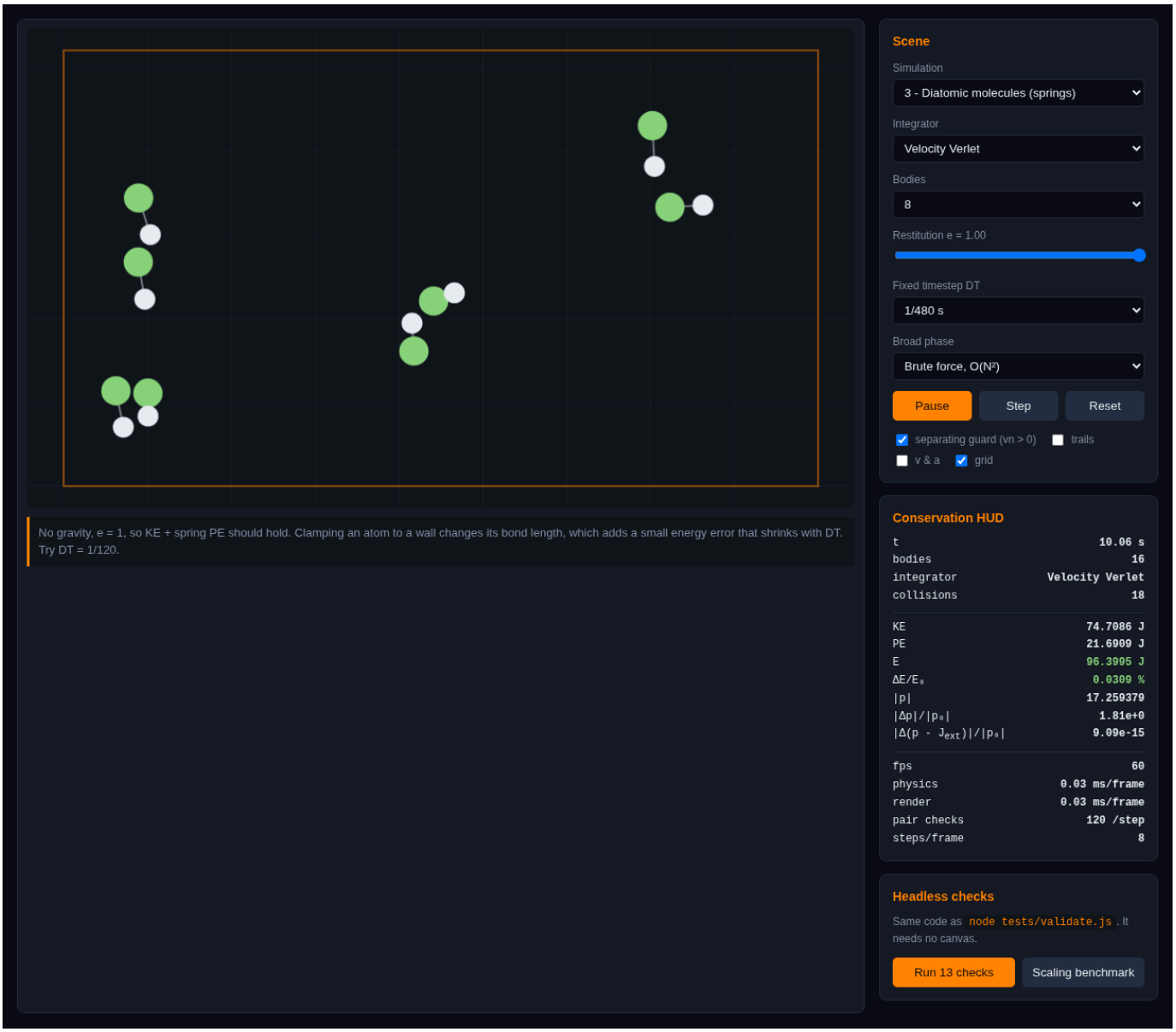


A star (m = 1), five planets and a moon, G = 1, softening 0.005, DT = 1/240 s. The star gets the opposite of the planets' momentum, so the center of mass stays still. My first version had the planets too close together, and one got thrown out to a very large r. I respaced them more.

Integrator	Max |ΔE/E0| Over 60 s	Largest Change in a Planet's Distance from the Star	|p|
Velocity Verlet	4.4e-7	4.6 %	3e-17
Euler-Cromer	9.4e-5	5.0 %	4e-18
Euler	2.2e-1	105 %	4e-18

Verlet keeps the energy bounded with no escape for 60 s. Euler drifts and the inner planets spiral out, like Lab 5. Momentum stays at round-off for every integrator, because every pull is applied as +F and −F.

Scene 3: Diatomic Molecules



Eight molecules, each a heavy atom (3 kg) and a light atom (1 kg) on a spring (k = 200 N/m, rest length 0.8 m). This is the HCl bond from my Block 1 checkpoint, now with many molecules colliding. Atoms in the same molecule share a `group`, so they do not collide with each other.

Diagnostic: No gravity or drag, and e = 1. So KE + spring PE should hold, and p − J_ext should hold because springs and pair impulses are internal.

DT	Max |ΔE/E0| over 60 s (Verlet)	|Δ(p − J_ext)|/|p0|
1/120	5.9e-2	5.4e-15
1/240	3.2e-2	1.4e-14
1/480	5.4e-3	9.6e-15

Momentum holds to round-off at every step. Energy does not hold as well at large DT. Clamping an atom to the wall moves it without moving its partner. That changes the bond length, which changes the spring energy. The atom overshoots the wall by less at smaller DT, so the error shrinks. This scene is set to default to DT = 1/480, where it should stay under 1%.

Performance Report

Scene 1, DT = 1/240 s, so 4 physics steps per frame at 60 fps. Measured with the Scaling benchmark button.

Bodies	Brute Checks/Step	Brute Physics (ms/frame)	Brute Render (ms/frame)	Hash Checks/Step	Hash Physics (ms/frame)	Hash Render (ms/frame)	fps, Brute	fps, Hash
20	190	0.10	0.37	1	0.07	0.17	60	60
50	1225	0.13	0.27	21	0.13	0.37	60	60
100	4950	0.67	0.33	36	0.60	0.43	60	60
200	19900	2.40	0.53	159	0.70	0.57	60	60
500	124750	21.57	1.27	384	2.63	1.03	44	60

Bottleneck: Physics is the bottleneck at large N. Render time grows slowly, from about 0.4 ms to 1.3 ms, while the body count grows 25 times. Brute-force physics grows much faster. From 200 to 500 bodies, O(N^2) predicts (500/200)^2 = 6.3 times more work. My time grew 9 times, a bit worse than N^2. That is probably because `bruteForcePairs` builds a new array of 124,750 pairs every step, which the garbage collector has to clear. At small N the timings are noisy, since each step takes only a fraction of a millisecond.

Largest Body Count at 60 fps: With brute force, 200 bodies take about 2.9 ms per frame, which is under the 16.7 ms budget. At 500 bodies it takes 22.8 ms, so the page drops to 44 fps. Scaling the physics time as N^2 puts the limit at roughly 420 bodies. The first thing to optimize is the broad phase.

Bonus: Spatial Grid Broad Phase

The hash puts bodies in cells of size 2 times the largest radius and only pairs bodies in neighboring cells. I sort its pairs into the same order brute force uses.

Bodies	Brute (ms/frame)	Hash (ms/frame)	Speedup	Checks/Step, Brute vs Hash
200	2.40	0.70	3.4 times	19900 vs 159
500	21.57	2.63	8.2 times	124750 vs 384

Physics: 200 balls for 5 seconds with each broad phase. Both had 2008 collisions, and the largest position difference was exactly 0. The grid does not change the physics.

Engine Changes a Scene Forced

No scene needed an engine edit in the end, but two engine features came from building scenes:

- `group` on Body came from Scene 3. Without it, the two atoms of a molecule would collide with each other.
- `isInternal` on forces came from the ball pit. The HUD showed a huge p − J_ext change there, because gravity also adds momentum from outside. Spring and NewtonGravity now mark themselves internal, and the HUD shows n/a when an outside force acts. World still never checks a force's type.